



Conocimiento (RAG) y Documentos

Dirigido a: **Administradores de tenant (tenant_admin)** y **operadores que gestionan el conocimiento que consumen los agentes**

Este manual explica cómo gestionar el **conocimiento** de la plataforma: las **Knowledge Bases (KBs)**, sus **categorías** y los **documentos** que se suben e indexan para alimentar el RAG (Retrieval-Augmented Generation) de los agentes.

Una Knowledge Base es un contenedor de documentos indexados sobre un modelo de embedding fijo. Los documentos se suben, se procesan (ingestión) y, una vez indexados, sus fragmentos quedan disponibles para que los agentes recuperen contexto. Las KBs se organizan en categorías y se conceden (grant) a proyectos y agentes, que son quienes finalmente las consumen.

Cubre las pantallas de listado y gestión de KBs, la gestión de categorías y el acceso a documentos a través de los proyectos, incluyendo la subida, el estado de indexación y el reindexado. La mayoría de acciones de creación, edición y borrado requieren rol **tenant_admin**.

Contenido

- 01** Listado de Knowledge Bases
- 02** Crear una Knowledge Base
- 03** Editar, conceder acceso (Grant) y borrar una KB
- 04** Ver asignaciones (proyectos y agentes con acceso)
- 05** Subir documentos a una KB e indexarlos
- 06** Reindexar, ver progreso y borrar documentos
- 07** Categorías de Knowledge Bases
- 08** Crear, editar y borrar categorías
- 09** Documentos por proyecto

1 Listado de Knowledge Bases

Esta es la pantalla principal de **Knowledge Bases** del tenant. Cada KB agrupa documentos indexados y se asigna (grant) a uno o varios proyectos. El listado se presenta **agrupado por categoría**: primero las categorías built-in (sembradas por la plataforma), luego las del tenant y, al final, un grupo **Sin categoría**; cada cabecera muestra el color de la categoría y el número de KBs.

- Cada KB se muestra como una tarjeta con su nombre, una etiqueta **Built-in** si procede, el modelo de embedding utilizado y su descripción.
- Al pulsar la fila (flecha a la izquierda) la tarjeta se **despliega** y aparece el panel de documentos de esa KB para subir y gestionar archivos sin salir de la pantalla.
- El botón **Asignaciones** abre un diálogo con los proyectos y agentes que tienen acceso a la KB.

Arriba a la derecha, con rol tenant_admin, dispone del botón **Categorías** (atajo a la gestión de categorías) y del botón **Crear KB**. Las KBs built-in son de solo lectura: solo permiten Grant y Asignaciones, sin editar ni borrar.



Figura 04.1 — Listado de Knowledge Bases

Crear una Knowledge Base

Pulse **Crear KB** (arriba a la derecha) para abrir el diálogo de creación. Una KB es un contenedor de documentos indexados; tras crearla se despliega en la lista para subir documentos y se le da acceso (grant) a los proyectos o agentes que la consumirán.

- **Nombre** (obligatorio, máximo 120 caracteres): identifica la KB en el listado.
- **Categoría** (opcional): un selector con las categorías built-in y las del tenant; sirve para organizar el listado. El botón + contiguo abre un mini-diálogo para crear una categoría nueva sin salir del flujo (pidiendo slug, nombre y color).
- **Descripción** (opcional): admite formato Markdown.

El **modelo de embedding** se asigna automáticamente al valor por defecto de la plataforma y no se puede cambiar después. Pulse **Crear KB** para guardar; si hay error se mostrará un mensaje en el propio diálogo.



Figura 04.2 — Crear una Knowledge Base

Editar, conceder acceso (Grant) y borrar una KB

Desde cada tarjeta del listado (con rol `tenant_admin`) dispone de varias acciones:

- **Grant:** abre el diálogo *Dar acceso a un proyecto*. Seleccione el proyecto destino con el buscador; tras conceder el acceso, ese proyecto verá la KB en su sub-sección Knowledge Bases y podrá subir documentos. Puede repetir el grant para varios proyectos.
- **Editar** (icono lápiz, solo en KBs no built-in): permite cambiar nombre, categoría y descripción. El modelo de embedding se muestra pero es de solo lectura: para usar otro modelo hay que crear una KB nueva y reindexar.
- **Borrar** (icono papelera, solo en KBs no built-in): acción **irreversible**. Borra la KB, todos sus documentos indexados y los grants a proyectos (los archivos en el almacenamiento de objetos no se tocan). Para confirmar debe teclear exactamente el nombre de la KB.

Las KBs built-in no muestran los botones de editar ni borrar, ya que el backend las protege como de solo lectura para el tenant.



AGENTIC PLATFORM

TRABAJO

RECURSOS

Agentes

Catálogo

Agentes humanos

Equipos

Proyectos

Knowledge Bases

Memorias

Documentos

CONFIGURACIÓN DEL TENANT

PLATAFORMA

AYUDA

Demo Manuales

SYSTEM_ADMIN

ES

EN

Demo Manuales Admin

Inicio

Knowledge Bases

Knowledge Bases

Bases de conocimiento del tenant. Cada KB agrupa documentos indexados y se asigna (grant) a uno o más proyectos.

Categorías

Crear KB

ROL (8)

CodeIgniter 4 — CI/CD y despliegue

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

CI/CD y despliegue: pipeline build -> test -> deploy, imagen Docker PHP-FPM + Nginx + supervisor, health-check, gate solo en main y checklist de deploy.

CodeIgniter 4 — Arquitectura HMVC y routing

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Arquitectura HMVC modular de CodeIgniter 4: estructura app/Modules/, patrón Config+Items, BaseEntity como MappedSuperclass, Second-Level Cache de Doctrine y routing config-driven.

CodeIgniter 4 — Modelo de datos con Doctrine

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Modelo de datos con Doctrine 3.x vía daycry/doctrine: attribute mapping, BaseEntity (UUID, timestamps, soft-delete, lifecycle), funciones JSON, migraciones reversibles, seeds y regiones de Second-Level Cache.

CodeIgniter 4 — Estrategia de testing

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Estrategia de testing con PHPUnit: suites Unit / Integration / E2E (Selenium/Chrome), modo estricto, phpunit.xml.dist y scripts composer de cobertura y mutación (Infection).

CodeIgniter 4 — Seguridad y autenticación

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Seguridad y autenticación con daycry/auth: authenticators session/JWT/access-token, grupos y permisos, rate-limiting, CSP/Cookies/CSRF y buenas prácticas de gestión de secretos.

CodeIgniter 4 — Internacionalización EN/ES

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Internacionalización EN/ES: defaultLocale=en, supportedLocales=['en','es'], ficheros de idioma de CodeIgniter 4, daycry/codeigniter-language, columnas JSON {es,en} y la UI de pestañas de idioma.

CodeIgniter 4 — Frontend y assets

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Frontend y pipeline de assets: JS core, TinyMCE, Select2, DataTables, Bootstrap, versionado de assets para cache-busting y macros Twig de formulario con campos traducibles por locale.

API REST design guidelines

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Principios de diseño de APIs REST: recursos, verbos HTTP, códigos de estado, versionado, paginación, filtrado, HATEOAS, y autenticación con OAuth 2 / JWT. Agnóstico de stack.

STACK (6)

CodeIgniter 4 — Convenciones del equipo

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Convenciones de stack para proyectos CodeIgniter 4: arquitectura HMVC, patrón Config+Items, estándares de código y toolchain, política i18n EN/ES y catálogo de dependencias del ecosistema (daycry/auth, daycry/doctrine, daycry/twig).

React + Next.js conventions

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Convenciones para frontend con Next.js 14 + React: app router, server components, TanStack Query, formularios con react-hook-form, y testing con vitest + Playwright.

Node + Express conventions

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Convenciones para APIs Node.js con Express: estructura de carpetas, middleware, validación con zod, testing con vitest, y patrones de conexión a Postgres con pg/Prisma.

PHP + Symfony conventions

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Convenciones de Symfony 6/7: controladores, autowiring, Doctrine, tests con phpunit, y migrations con doctrine-migrations.

PostgreSQL best practices

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Buenas prácticas con PostgreSQL: diseño de esquema, índices, JSONB, row-level security, conexiones, vacuum/autovacuum, y migraciones reversibles.

Python + FastAPI conventions

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Figura 04.3 — Editar, conceder acceso (Grant) y borrar una KB

Ver asignaciones (proyectos y agentes con acceso)

El botón **Asignaciones** de cada KB abre un diálogo que muestra a quién se ha concedido acceso, en dos secciones:

- **Proyectos:** lista de proyectos con grant. Cada fila tiene un botón de papelera para **revocar** el acceso de ese proyecto.
- **Agentes:** lista de agentes con grant, mostrando su rol y su ámbito (scope) mediante etiquetas. Cada fila permite revocar el acceso del agente.

Si la KB no está concedida a nadie, el diálogo lo indica y recuerda cómo conceder acceso: a un proyecto con el botón **Grant** de la lista, y a un agente desde su detalle (pestaña Knowledge Bases → Grant KB). Estas asignaciones son las que determinan qué agentes pueden recuperar el conocimiento de la KB durante el RAG.



Figura 04.4 — Ver asignaciones (proyectos y agentes con acceso)

Subir documentos a una KB e indexarlos

Para gestionar documentos, despliegue una KB en el listado (pulsando su fila) para abrir su **panel de documentos**. El panel muestra el número de documentos y un botón **Subir documento** (rol tenant_admin).

- El diálogo de subida acepta un **Archivo** (formatos admitidos: .pdf, .docx, .md, .txt, .html, .wav, .mp3) y un **Título** opcional (por defecto se usa el nombre del archivo).
- Al subir, el documento entra en la cola de **ingestión** y se procesa para extraer su texto y generar los fragmentos (chunks) e índices que usará el RAG.

Cada documento muestra un **estado** mediante una etiqueta de color: **Pendiente**, **Procesando**, **Indexado** (verde, listo para RAG), **Indexado vacío** (aviso: se procesó pero con 0 fragmentos, por lo que el agente no podrá recuperar nada; conviene subir un original con texto seleccionable o reindexar) y **Fallido** (con su mensaje de error). También se ven el tipo MIME, el tamaño y el número de páginas.



Figura 04.5 — Subir documentos a una KB e indexarlos

Reindexar, ver progreso y borrar documentos

En el panel de documentos de cada KB, dentro de la fila de cada documento, dispone de acciones según su estado:

- **Progreso:** enlace que abre la página de detalle de ingestión del documento, donde puede seguir el avance del procesamiento.
- **Reindexar** (icono de recarga, rol tenant_admin): vuelve a procesar el documento. Está disponible cuando el documento ya terminó (estados Indexado, Indexado vacío o Fallido); útil para reintentar un fallo o reprocesar tras corregir el original. Se oculta mientras el documento está Pendiente o Procesando.
- **Eliminar** (icono papelera, rol tenant_admin): borra el documento de la KB.

Cuando un documento queda **Indexado vacío** se muestra un aviso explicando que no tiene fragmentos recuperables. El estado se refresca al consultar la lista, por lo que conviene volver a desplegar la KB para ver la progresión de Procesando a Indexado.



AGENTIC PLATFORM

TRABAJO

RECURSOS

Agentes

Catálogo

Agentes humanos

Equipos

Proyectos

Knowledge Bases

Memorias

Documentos

CONFIGURACIÓN DEL TENANT

PLATAFORMA

AYUDA

Demo Manuales

SYSTEM_ADMIN

ES

EN

D Demo Manuales Admin demo@example.com

Inicio

Knowledge Bases

Knowledge Bases

Bases de conocimiento del tenant. Cada KB agrupa documentos indexados y se asigna (grant) a uno o más proyectos.

Categorías

Crear KB

ROL (8)

CodeIgniter 4 — CI/CD y despliegue

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

CI/CD y despliegue: pipeline build -> test -> deploy, imagen Docker PHP-FPM + Nginx + supervisor, health-check, gate solo en main y checklist de deploy.

CodeIgniter 4 — Arquitectura HMVC y routing

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Arquitectura HMVC modular de CodeIgniter 4: estructura app/Modules/, patrón Config+Items, BaseEntity como MappedSuperclass, Second-Level Cache de Doctrine y routing config-driven.

CodeIgniter 4 — Modelo de datos con Doctrine

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Modelo de datos con Doctrine 3.x vía daycry/doctrine: attribute mapping, BaseEntity (UUID, timestamps, soft-delete, lifecycle), funciones JSON, migraciones reversibles, seeds y regiones de Second-Level Cache.

CodeIgniter 4 — Estrategia de testing

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Estrategia de testing con PHPUnit: suites Unit / Integration / E2E (Selenium/Chrome), modo estricto, phpunit.xml.dist y scripts composer de cobertura y mutación (Infection).

CodeIgniter 4 — Seguridad y autenticación

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Seguridad y autenticación con daycry/auth: authenticators session/JWT/access-token, grupos y permisos, rate-limiting, CSP/Cookies/CSRF y buenas prácticas de gestión de secretos.

CodeIgniter 4 — Internacionalización EN/ES

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Internacionalización EN/ES: defaultLocale=en, supportedLocales=[en,es], ficheros de idioma de CodeIgniter 4, daycry/codeigniter-language, columnas JSON {es,en} y la UI de pestañas de idioma.

CodeIgniter 4 — Frontend y assets

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Frontend y pipeline de assets: JS core, TinyMCE, Select2, DataTables, Bootstrap, versionado de assets para cache-busting y macros Twig de formulario con campos traducibles por locale.

API REST design guidelines

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Principios de diseño de APIs REST: recursos, verbos HTTP, códigos de estado, versionado, paginación, filtrado, HATEOAS, y autenticación con OAuth 2 / JWT. Agnóstico de stack.

STACK (6)

CodeIgniter 4 — Convenciones del equipo

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Convenciones de stack para proyectos CodeIgniter 4: arquitectura HMVC, patrón Config+Items, estándares de código y toolchain, política i18n EN/ES y catálogo de dependencias del ecosistema (daycry/auth, daycry/doctrine, daycry/twig).

React + Next.js conventions

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Convenciones para frontend con Next.js 14 + React: app router, server components, TanStack Query, formularios con react-hook-form, y testing con vitest + Playwright.

Node + Express conventions

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Convenciones para APIs Node.js con Express: estructura de carpetas, middleware, validación con zod, testing con vitest, y patrones de conexión a Postgres con pg/Prisma.

PHP + Symfony conventions

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Convenciones de Symfony 6/7: controladores, autowiring, Doctrine, tests con phpunit, y migrations con doctrine-migrations.

PostgreSQL best practices

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

Buenas prácticas con PostgreSQL: diseño de esquema, índices, JSONB, row-level security, conexiones, vacuum/autovacuum, y migraciones reversibles.

Python + FastAPI conventions

Built-in

Asignaciones

Grant

embedding: nomic-embed-text-v1.5

AGENTIC PLATFORM

Conocimiento (RAG) y Documentos

pág. 25 / 28

Figura 04.6 — Reindexar, ver progreso y borrar documentos

7 Categorías de Knowledge Bases

Esta pantalla gestiona las **categorías** que agrupan las KBs en el listado. Se accede desde el botón **Categorías** de la pantalla de Knowledge Bases o desde la migaja de pan superior.

- **Built-in:** la plataforma siembra 5 categorías comunes a todos los tenants (stack, role, compliance, architecture, process). Son de **solo lectura**, se marcan con la etiqueta Built-in y no muestran botones de editar ni borrar.
- **Tenant:** las categorías propias que cree el tenant. Si no hay ninguna, se indica que puede usar las built-in o crear una nueva.

Cada categoría se muestra como una tarjeta con su color, su nombre y su slug (identificador estable). Con rol tenant_admin, en las categorías propias aparecen los botones de editar y borrar. Arriba a la derecha está el botón **Nueva categoría**.

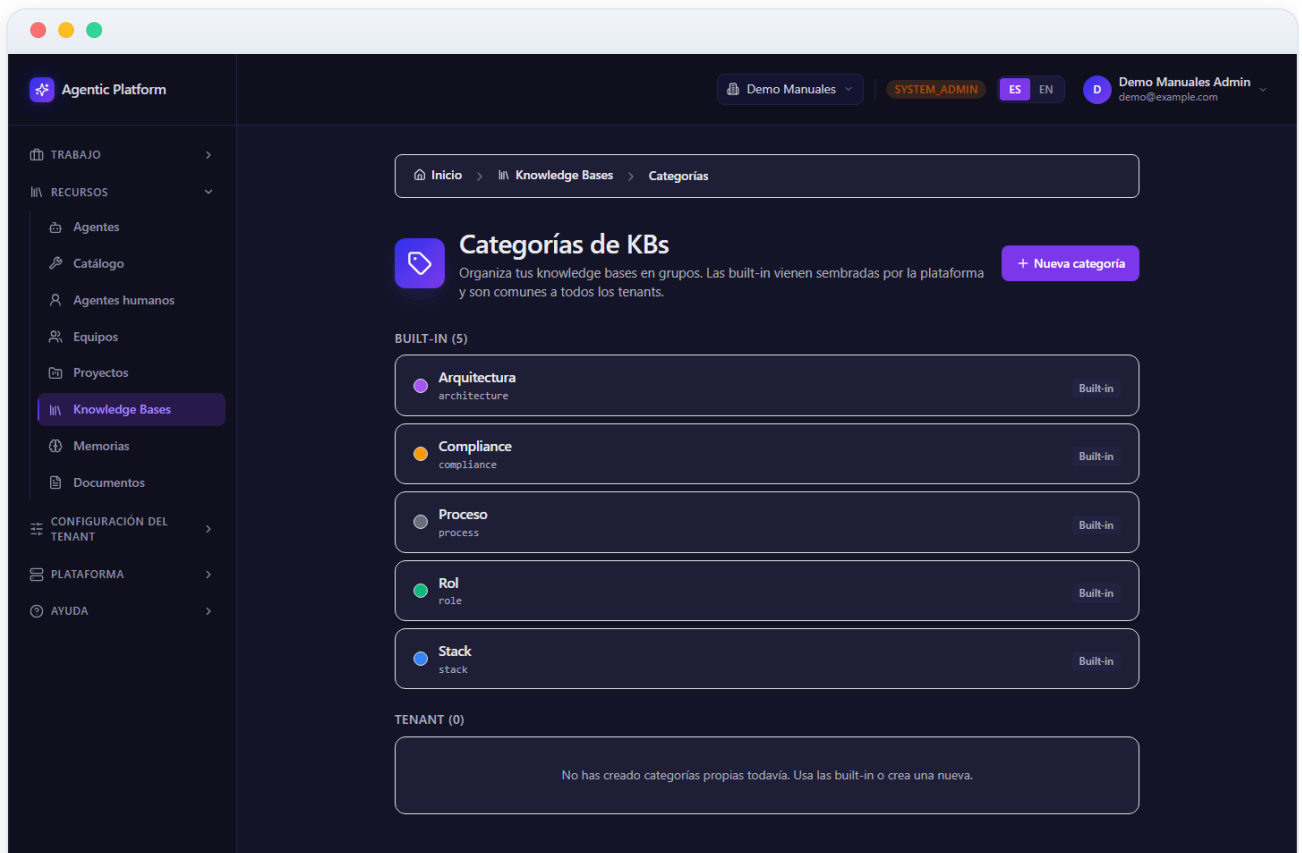


Figura 04.7 — Categorías de Knowledge Bases

8 Crear, editar y borrar categorías

Pulse **Nueva categoría** para abrir el diálogo de creación. Debe indicar:

- **Slug:** identificador estable usado en filtros y URLs; solo admite minúsculas, números, guion y guion bajo (p. ej. `compliance-pci`).
- **Nombre:** el texto que se muestra en la interfaz (p. ej. *Compliance PCI-DSS*).
- **Color:** selector de color (o código hexadecimal) que identifica visualmente la categoría en el listado de KBs.

Al **editar** una categoría propia puede cambiar el nombre y el color, pero **no el slug** (es fijo porque puede estar referenciado en filtros e integraciones). Al **borrar** una categoría, las KBs que pertenecían a ella no se eliminan: simplemente pasan a quedar **Sin categoría**. Las categorías built-in no se pueden editar ni borrar.

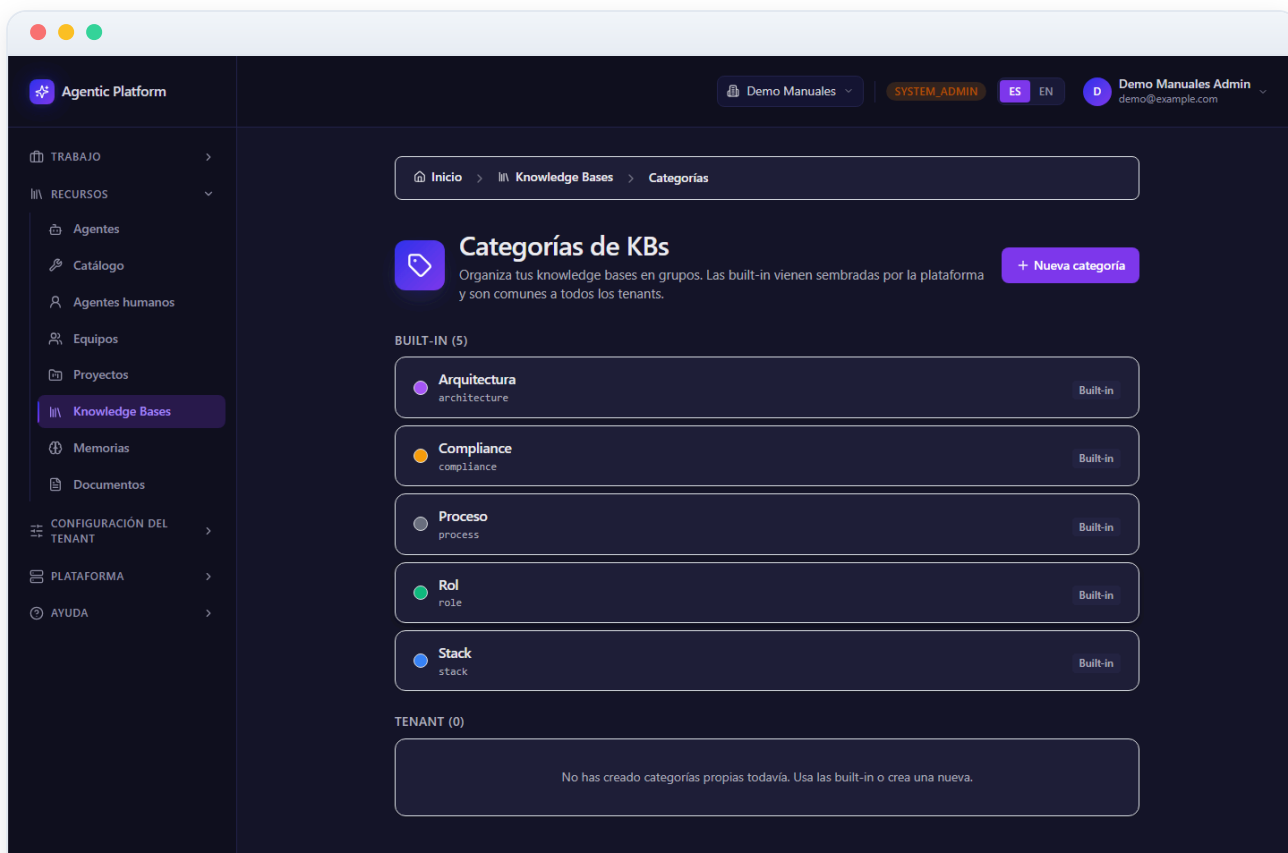


Figura 04.8 — Crear, editar y borrar categorías

9 Documentos por proyecto

La sección **Documentos** es el punto de entrada para localizar documentos a través de los proyectos. El sistema no mantiene un listado global de documentos: cada documento vive dentro de una Knowledge Base de un proyecto, por lo que esta pantalla muestra una rejilla con todos los **proyectos** del tenant.

- Cada tarjeta muestra el nombre del proyecto y su descripción.
- Al hacer clic en un proyecto se navega a su sección **Knowledge Bases**, desde donde se accede a las KBs concedidas a ese proyecto y a sus documentos.

Si el tenant aún no tiene proyectos, la pantalla lo indica e invita a crear uno desde </admin/projects/new>. Recuerde que para que un proyecto vea documentos primero debe tener una KB concedida mediante Grant desde la pantalla de Knowledge Bases.

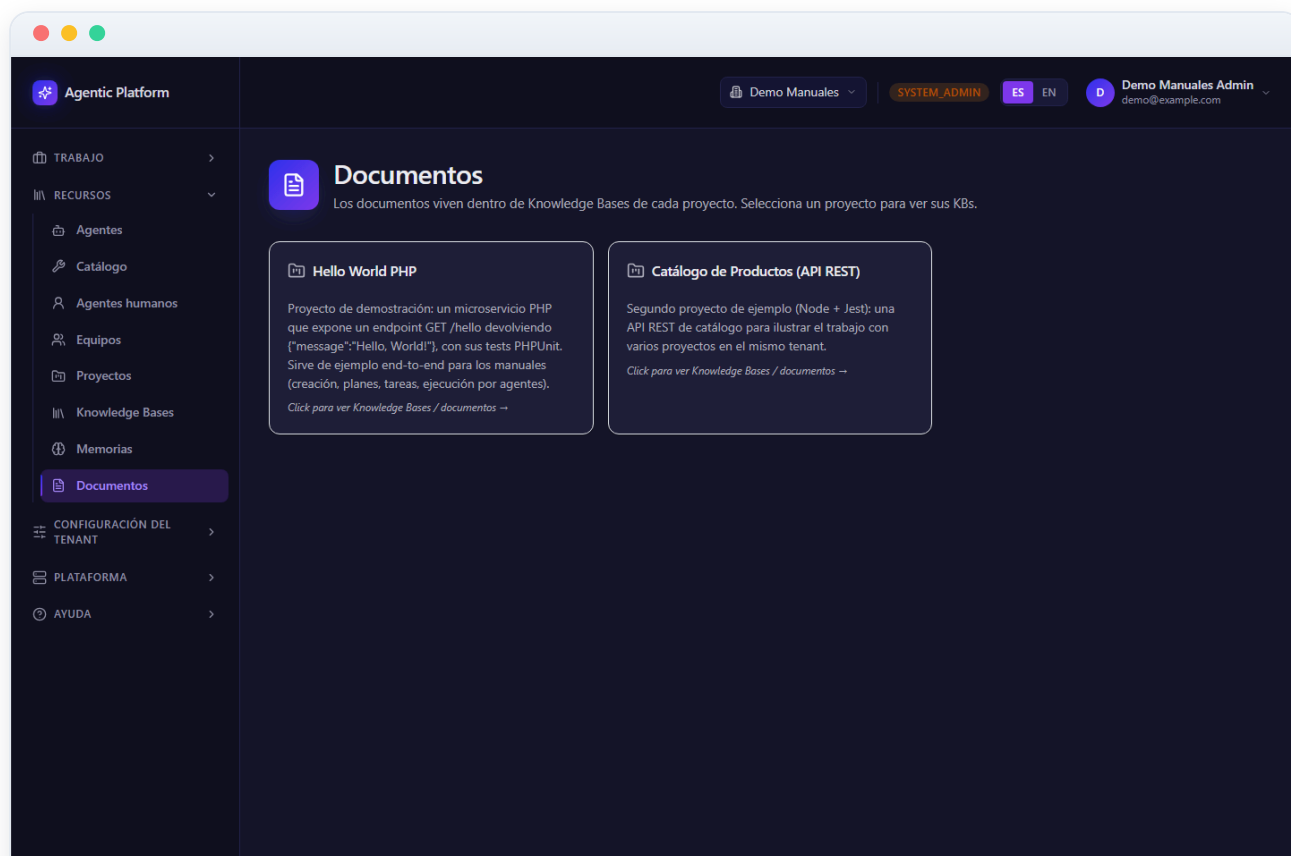


Figura 04.9 — Documentos por proyecto